Decisiones de Diseño para cada problema resuelto:

4.1. Qué ocurre a continuación: Agenda de eventos del reloj virtual

1. Problema

En el juego el tiempo es virtual, no pasa solo, sino que salta de un evento al siguiente. Estos eventos son acontecimientos que van a pasar en un momento determinado, ya sea la próxima acción del jugador o de un enemigo, una puerta que se cierra sola, un golpe de veneno, el final de una poción de velocidad, una antorcha que se apaga o una trampa que se vuelve a armar.

Siempre se necesita saber cuál es el evento que sigue, o sea, el que tiene el tiempo más cercano y si dos empatan, el cercano es el que se programó primero. Después de ejecutarlo, casi siempre hay que programar otro, además de que hay que poder cancelar un evento (si muere un enemigo o el jugador usa un antídoto) y cambiarle el tiempo (si cambia la velocidad). Todo esto teniendo siempre que dar el mismo orden.

Sabiendo esto, se mide cuánto se usa cada operación con **bench/bench_agenda.py**, en un mundo de 100 salas y 300 enemigos. Por cada acción del jugador se programaron 301 eventos y se sacaron 301, con un máximo de 300 pendientes a la vez. Cancelar y cambiar el tiempo dieron 0 porque en esa prueba nadie murió ni cambió de velocidad, son cosas que es muy poco probable que pase pero puede pasar. Entonces lo que más ocurre es sacar el evento que sigue y meter uno nuevo, más o menos una vez por cada personaje en cada ciclo.

2. Alternativas consideradas

- **Lista sin ordenar:** meter un evento es rápido, pero para saber cuál sigue hay que revisarlos todos. Se descarta porque eso se hace en cada paso.
- **Lista siempre ordenada:** saber cuál sigue es rápido, pero cada vez que se mete uno hay que hacerle espacio moviendo otros. Se descarta porque por cada evento que se saca se mete otro.
- **heapq de Python:** se descartó porque el proyecto pide que se hagan estructuras propias y además no deja cancelar un evento cualquiera de forma sencilla.

3. Decisión

Se hizo un **min heap** propio, guardado en una lista, ya que es muy funcional porque el evento más cercano siempre queda arriba. Por eso sacarlo es rápido, y acomodar los demás después cuesta poco.

Para comparar eventos usamos las variables **tiempo**, **número de secuencia**. El número de secuencia sube con cada evento nuevo y resuelve los empates: si dos eventos tienen el mismo tiempo, siempre sale primero el que se programó antes, así el orden nunca cambia. Cada evento recuerda en qué posición de la lista está y gracias a eso podemos cancelarlo o cambiarle el tiempo sin buscarlo entre todos.

El tiempo de la próxima acción de un personaje se calcula con **intervalo = max(1, costo * 100 // velocidad)** que mientras más rápido es, menos espera. Si su velocidad cambia mientras espera, se ajusta el tiempo que le falta para no perder lo que ya había avanzado.

Cada cambio en la agenda deja anotado cómo deshacerlo, y así el pergamino de retroceso puede volver atrás sin guardar copias de todo.

4. Análisis de complejidad

Operación	**Heap**	**Lista sin ordenar**	**Lista ordenada**
Programar un evento	O(log n)	O(1)	O(n)
Sacar el que sigue	O(log n)	O(n)	O(1)
Cancelar o cambiar el tiempo	O(log n)	O(n)	O(n)
Ver cuál sigue	O(1)	O(n)	O(1)

Como en cada ciclo se saca y se mete casi un evento por personaje, lo que importa es la suma de las dos operaciones. En el heap es O(log n). En las dos listas es O(n), porque una de las dos operaciones siempre cuesta mucho. Cancelar es O(log n) en el heap porque cada evento sabe dónde está, en las listas primero hay que buscarlo.

5. Evidencia empírica

Se utiliza **bench/bench_agenda.py** para medirlo en microsegundos por operación.

```
Extraer el próximo y programar otro (microsegundos por operación):
 pendientes               heap   lista sin ordenar      lista ordenada
        100                3.3                 6.5                 9.1
        300                4.0                18.3                24.3
       1000                4.3                61.1                73.2
       3000                6.4               190.6               140.0

Cancelar un evento cualquiera y programar otro (microsegundos):
 pendientes               heap   lista sin ordenar      lista ordenada
        100                2.5                 1.9                 8.4
        300                2.3                 3.6                23.2
       1000                2.4                12.4                91.1
       3000                2.6                39.5               258.3
```

Basándonos en las dos cosas que más hace la agenda: sacar el evento que sigue y programar otro, y cancelar un evento cualquiera y programar otro.

En las dos pruebas el heap casi no cambia, aunque haya muchos más eventos (entre 2.3 y 6.4 microsegundos con 100 hasta 3000 pendientes), mientras que las listas se van poniendo cada vez más lentas (con 3000 pendientes llegan a 190 microsegundos al sacar y 258 microsegundos al cancelar).

Con 300 eventos pendientes, un ciclo hace unas 300 de estas operaciones, así que el heap tarda entre 0.7 y 1.2 ms. Con 1000 eventos, el heap tarda entre 2.4 y 4.3 ms, mientras que las listas tardan entre 12 y 91 ms, y casi siempre pasan el límite de 20 ms por ciclo del enunciado. (Son estimaciones, dependen de la computadora y de cuántos eventos se muevan en cada ciclo.)

Hay un caso donde una alternativa ganó, al cancelar con 100 eventos pendientes, la lista sin ordenar tardó 1.9 y el heap 2.5 microsegundos. La diferencia es menos de un microsegundo, y cancelar pasa poco. Desde 300 eventos el heap ya gana.

Para comprobar esas estimaciones se midió el ciclo completo del juego (esperar y retroceder, 3000 acciones por escenario). Con 100 salas y 300 enemigos, el ciclo promedió 1,7 ms, con percentil 99 de 2,2 ms y peor caso de 5,1 ms. Con 20 salas y 30 enemigos, el percentil 99 fue de 0,23 ms. Esto es del mismo orden que la estimación del heap (0,7 a 1,2 ms con 300 eventos), un poco mayor porque el ciclo completo también incluye los enemigos, el historial y el retroceso, y queda unas 9 veces por debajo del límite de 20 ms. Son mediciones en mi computadora; otro equipo dará valores distintos.

```
 escenario  salas  enemigos   promedio       p99      peor   (ms)
    normal     20        30      0.403     1.303     3.898   cumple
    grande    100       100      0.953     2.412     5.604   cumple
   extremo    100       300      6.234    14.912    61.096   cumple

Limite: 20 ms por ciclo (veredicto sobre el percentil 99)

Process finished with exit code 0
```

6. Límites conocidos

- Si la agenda siempre fuera muy chica con unos 100 eventos o menos, una lista simple alcanzaría. El heap vale la pena porque, desde unos cientos de eventos, las listas se vuelven demasiado lentas.
- El heap no deja recorrer los eventos en orden sin sacarlos y si quisiéramos mostrar la agenda completa, habría que copiarla y ordenarla, lo que cuesta O(n log n).
- Cancelar rápido depende de que cada evento guarde su posición. Eso une al evento con la estructura y obliga a mantener esa posición actualizada cada vez que algo se mueve.
- Los tiempos dependen de la computadora y de la versión de Python. Lo que debería mantenerse en otro equipo son las proporciones entre las estructuras, no los números exactos.

Efectos aparte (veneno y regeneración): las fichas de la cripta pueden traer un valor de veneno en las trampas o de regeneración en los enemigos, o no traerlo. El programa los lee con un valor por defecto de 0: si la ficha los trae, la trampa envenena o el enemigo se regenera mediante eventos programados en la agenda, si no, no ocurre nada. Así no hay valores escritos a mano en el código y el juego funciona con cualquier cripta, incluida una que no se haya visto antes. Se verificó con pruebas automáticas para ambos casos (con y sin el campo).

4.2 Traer el contenido antes que haga falta

1. **Problema**

Para entrar a una sala hay que conocer sus enemigos, objetos y trampas. Cada solicitud tarda unos 400 ms y gasta una unidad de presupuesto_solicitudes, que cambia de una cripta a otra. El servicio entrega hasta 10 salas por solicitud al mismo costo. La operación que el programa ejecuta de verdad es una por cada sala nueva a la que entra el jugador (FabricaModelo.poblar_sala llama a RepositorioCripta.precargar antes de confirmar el movimiento). En una caminata de

150 pasos sobre 100 salas hay unas 84 entradas nuevas; las entradas a salas ya vistas no piden nada.

2. **Alternativas consideradas**

- **Una sala por solicitud:** Pide solo la sala de destino
- **Anticipar en cada sala:** Al entrar a cada sala nueva pide un lote de 10 salas cercanas, aunque el destino ya se conozca
- **Pedir por lotes solo si falta:** Pide un lote de 10 salas cercanas solo cuando falta el contenido del destino

3. **Decisión**

El programa pide el contenido de las salas en lotes de 10 y solo cuando falta el de la sala a la que va a entrar el jugador; con eso hace entre 5 y 8 veces menos solicitudes que pedir sala por sala. Si el contenido del destino ya está en contenidos, no se pide nada. Si falta, precargar recorre las salas en anchura desde el destino con una cola propia, reune las 10 salas más cercanas cuyo contenido aún no llegó (el destino incluido) y las pide en una sola solicitud. contenidos es una lista paralela al VectorBusqueda de salas y el marcado de visitadas usa una lista de booleanos, sin dict ni set. La solicitud es síncrona y ocurre antes de partida.mover, por eso el jugador nunca queda en una sala sin contenido y el tiempo de red no toca el reloj virtual. Una sala ya pedida no se vuelve a pedir porque la condición es contenidos[pos] is None.

4. **Análisis de complejidad**

Cada llamada a precargar cuesta O(n) para crear la lista de visitadas, más el recorrido en anchura sobre las V salas exploradas hasta juntar 10 desconocidas, con O(log(n)) por cada búsqueda de posición. Pero solo se ejecuta cuando falta el contenido: en las pruebas, una de cada cinco entradas nuevas. En red, cada solicitud trae 10 salas nuevas, así que toda la cripta de n salas cuesta como máximo [n/10] solicitudes de contenido.

5. **Evidencia empírica**

Cripta y caminata	**Una sala por solicitud**	**Anticipar en cada sala**	**Lote solo si falta**
100 salas, 150 pasos	**83,5**	**10,0**	**9,9**
900 salas, 40 pasos	**35,2**	**35,2**	**7,1**

900 salas, 200 pasos	**147,8**	**90,0**	**25,8**
2500 salas, 100 pasos	**80,5**	**80,5**	**14,7**

```
Cripta de 100 salas, caminata de 150 pasos (promedio de 20 caminatas):
              estrategia  solicitudes  salas pedidas  salas visitadas
  una sala por solicitud         83.5           83.5             83.5
  anticipar en cada sala         10.0          100.0             83.5
      lote solo si falta          9.9           99.0             83.5

Cripta de 900 salas, caminata de 40 pasos (promedio de 20 caminatas):
              estrategia  solicitudes  salas pedidas  salas visitadas
  una sala por solicitud         35.2           35.2             35.2
  anticipar en cada sala         35.2          352.5             35.2
      lote solo si falta          7.1           71.0             35.2

Cripta de 900 salas, caminata de 200 pasos (promedio de 20 caminatas):
              estrategia  solicitudes  salas pedidas  salas visitadas
  una sala por solicitud        147.8          147.8            147.8
  anticipar en cada sala         90.0          900.0            147.8
      lote solo si falta         25.8          257.5            147.8

Cripta de 2500 salas, caminata de 100 pasos (promedio de 20 caminatas):
              estrategia  solicitudes  salas pedidas  salas visitadas
  una sala por solicitud         80.5           80.5             80.5
  anticipar en cada sala         80.5          804.5             80.5
      lote solo si falta         14.7          146.5             80.5
```

6. **Límites conocidos**
   - La solicitud es síncrona: el jugador espera unos 400 ms al entrar a una sala sin contenido, aproximadamente una de cada cinco entradas nuevas. Esconder esa espera exigiría precargar en segundo plano; no se hizo, y por eso en el diseño no existen respuestas pendientes ni concurrentes.
   - El contenido de las salas no se guarda en disco entre ejecuciones; solo se persisten las fichas del catálogo. Una partida nueva sobre la misma cripta vuelve a pedirlo.
   - La lista de visitadas se crea completa en cada llamada. Con criptas de cientos de miles de salas habría que reutilizarla.

4.3 Resolución de referencias del catálogo

1. Problema

Antes de crear un enemigo, objeto o trampa hay que convertir su id de tipo en una ficha. Los ids aparecen en el orden en que llegan las salas, se repiten entre salas y pueden estar ya en memoria, en disco o en ninguna parte. La operación real es RepositorioCatalogo.resolver(ids): una vez por cada lote de salas que llega y una vez más por cada sala que se puebla (esta última casi siempre acierta en memoria o en disco). En nuestras pruebas una sala trae unos 4 ids y un lote de 10 salas, unos 40, con muchos repetidos.

2. Alternativas consideradas

- **Un id por solicitud:** Una solicitud por cada tipo nuevo
- **Por sala:** Resuelve los ids de cada sala al poblarla
- **Por lotes de salas:** Resuelve de una vez los ids de todas las salas que llegan en un lote

3. Decisión

Los ids de tipo se resuelven en lotes de 10 y por lote de salas, no por sala ni por id; así el catálogo cuesta entre dos y tres veces menos solicitudes que resolver sala por sala. (resolver) por lotes de sala hace cuatro pasos. Primero, la función faltantes quita los ids repetidos con un VectorBusqueda ordenado y aparta los que ya están en memoria. Segundo, los pendientes se buscan en disco (AlmacenFichas, un archivo JSON por ficha). Tercero, los que no están en ningún lado se piden a la red en lotes de 10 y se guardan en disco apenas llegan. Cuarto, las fichas de enemigos traen la lista de objetos que sueltan, y esos ids se resuelven en una segunda pasada para que la muerte de un enemigo no genere una solicitud en pleno combate. La versión del catálogo se consulta una vez por ejecución y, si cambió, se borra lo guardado. El programa llama a resolver con los ids de todo el lote de salas apenas llega, y luego con los de la sala que se va a poblar. Si una ficha fue expulsada de memoria entre ambos pasos (la caché tiene tope), FabricaModelo._ficha la recupera del disco en vez de fallar.

4. Análisis de complejidad

Sea n la cantidad de ids de la llamada, c el tamaño de la caché y m los tipos que faltan. Quitar repetidos cuesta $O(n^2)$ en el peor caso, porque insertar_ordenado desplaza elementos; con n de unas decenas es despreciable. Preguntar si una ficha está en memoria cuesta $O(\log c)$. Cada ficha en disco cuesta una lectura. En red, las solicitudes son $\lceil m/10 \rceil$ por llamada, y en total no pasan de $\lceil T/10 \rceil$ más una de versión, donde T son los tipos distintos de la cripta.

5. Evidencia empírica

Solicitudes de catálogo con caché de 25 fichas, promedio de 10 caminatas aleatorias (bench_catalogo, semillas fijas; no dependen de la velocidad del equipo):

Cripta, pasos y tipos	Un id por solicitud	Por sala	Por lote de salas
100 salas, 150 pasos, 30 tipos	26,7	17,8	5,6
900 salas, 80 pasos, 30 tipos	26,9	17,2	6,4
900 salas, 200 pasos, 60 tipos	32,4	22,7	12,2

```
Cripta de 100 salas, 150 pasos, 30 tipos distintos (promedio de 10 caminatas, caché de 25):
            estrategia   solicitudes de catálogo  tipos vistos
   un id por solicitud                     26.7          26.7
              por sala                     17.8          26.7
     por lote de salas                      5.6          26.7

Cripta de 900 salas, 80 pasos, 30 tipos distintos (promedio de 10 caminatas, caché de 25):
            estrategia   solicitudes de catálogo  tipos vistos
   un id por solicitud                     26.9          26.9
              por sala                     17.2          26.9
     por lote de salas                      6.4          26.9

Cripta de 900 salas, 200 pasos, 60 tipos distintos (promedio de 10 caminatas, caché de 25):
            estrategia   solicitudes de catálogo  tipos vistos
   un id por solicitud                     32.4          32.4
              por sala                     22.7          32.4
     por lote de salas                     12.2          32.4
```

6. Limites conocidos

- Al resolver el lote completo se piden fichas de salas que el jugador quizá nunca visite. Se acepta porque la solicitud es el recurso escaso, pero con un presupuesto casi agotado convendría resolver solo lo necesario.
- Si un lote trae más tipos distintos que el tope de la caché, las primeras fichas se expulsan y se vuelven a leer del disco; las lecturas de disco suben más del doble, pero cada una cuesta unos 100 µs frente a unos 400 ms de una solicitud.
- Quitar repetidos es $O(n^2)$; con miles de ids por llamada habría que ordenarlos con el mergesort y recorrerlos una vez.
- La versión del catálogo se verifica solo al empezar: un cambio de versión a mitad de la partida no se detecta.

4.4 Reutilización y caché persistente

1. **Problema**

Preguntar si una ficha está en memoria ocurre cada vez que se crea un enemigo, objeto o trampa, y marcar_en_uso se ejecuta tras cada acción del jugador que cambia el estado (mover, atacar, esperar, recoger, soltar, usar, retroceder). Tras cada una, el programa recorre las fichas de los enemigos vivos, de los objetos del inventario y del suelo de la sala, y de las trampas de la sala. El tope por defecto es 25 fichas y se cambia con --cache-size. Una ficha en uso no puede salir de memoria.Alternativas consideradas

2. **Alternativas consideradas:**

Decisión	**Alternativa descartada**	**Resultado medido**	**Razón**
Cómo buscar	Recorrer listas paralelas	Con 25 000 fichas: 2,2 µs contra 681 µs	A 25 fichas casi no hay diferencia (0,57 contra 0,82 µs), pero el tope es configurable
Qué expulsar	FIFO: sale la más antigua en llegar	Con zona repetida y tope 25: 69,2 % de aciertos LRU contra 59,5 % FIFO	El jugador vuelve a las mismas salas, y LRU conserva lo que se usa de nuevo
Fichas en uso	Eventos precisos: el modelo avisa a la caché cuando nace o muere una instancia	No medida; se descartó por acoplar el modelo a la caché	Habría que deshacer esas llamadas al retroceder; recalcular tras cada acción es más simple y el costo es bajo

3. **Decisión**

La caché en memoria es un vector ordenado de ids más una lista doble que lleva el orden de uso: buscar cuesta O(log n), reordenar cuesta O(1), y cuando se llena sale la ficha menos usada recientemente que no esté en uso. Lo que sale se recupera del disco en unos 100 µs, no de la red. CacheFichas guarda un VectorBusqueda (ids ordenados, búsqueda binaria) y una ListaDoble donde la más reciente está al frente. Cada nodo tiene un contador en_uso. Al llenarse, se recorre

la lista desde el final y sale el primer nodo con contador 0. Si todas están en uso, la caché supera su tope en vez de romper una referencia.

marcar_en_uso reconcilia: primero suma las fichas del estado nuevo y después resta las del anterior, así una ficha que sigue en uso nunca pasa por cero. Lo que sale de memoria queda en cache/ (un JSON por ficha) y la versión del catálogo se compara una vez por ejecución contra /catalogo/version. Un acierto es una ficha ya en memoria o en disco; un fallo, una ficha que hubo que pedir a la red.

4. Analisis de complejidad

Sea c la cantidad de fichas en memoria y k las fichas distintas en uso. contiene y obtener: O(log c). Reordenar: O(1). Insertar o eliminar en el vector: O(c) por el desplazamiento, pero solo ocurre en un fallo de memoria. Liberar: O(c) en el peor caso, si hay que saltar nodos en uso. marcar_en_uso: O((k + k′) log c) por acción del jugador.

5. Evidencia empírica

Origen	Costo por ficha
Memoria	3 µs
Disco	unos 100 µs
Red	400 ms por solicitud, 40 ms por ficha con un lote lleno

```
Buscar una ficha por id (microsegundos y comparaciones por búsqueda):
  fichas  binaria us  lineal us  binaria cmp  lineal cmp
     25        2.46       3.15          4.0        13.0
    250        4.15      25.51          7.0       125.8
   2500        6.96     285.77         10.4      1250.2
  25000        9.82    3061.87         13.7     12450.9

Tasa de aciertos con accesos que repiten zona (80% a 20 fichas, 300 fichas en total):
 capacidad    LRU %   FIFO %
        10     32.2     30.5
        25     69.2     59.5
        50     83.3     75.0

Tasa de aciertos con accesos sin repetición de zona (uniformes sobre 300 fichas):
 capacidad    LRU %   FIFO %
        10      3.4      3.4
        25      8.6      8.6
        50     17.0     17.0

binaria = VectorBusqueda (lo que hace el programa), lineal = recorrer las listas paralelas
LRU     = CacheFichas (lista doble + vector), FIFO = expulsar la más antigua en llegar
```

Reproduciendo con --replay la misma partida de la cripta-04 (presupuesto de 52 solicitudes) con la caché limitada a 3 fichas (--cache-size 3, el caso más exigente), con la caché de cache/ vacía el programa gastó 8 solicitudes: 10 fichas tuvieron que pedirse a la red, en lotes. Con la caché caliente gastó 5, tres menos, sin ningún fallo: las 43 consultas salieron de memoria o de disco.

Costo de marcar_en_uso por acción del jugador, con la mitad de las fichas cambiando (referencia: 20 ms):

Fichas distintas en uso	**Costo por acción**	**% de 20 ms**
5	10,5 µs	0,05 %
25	79 µs	0,4 %
100	417 µs	2,1 %
400	2,5 ms	12,5 %

Búsqueda por id y tasa de aciertos (bench_cache):

Fichas	**Binaria**	**Lineal**
25	0,57 µs	0,82 µs
250	1,0 µs	6,1 µs
2 500	1,6 µs	67 µs
25 000	2,2 µs	681 µs

Tope de caché	**LRU con zona repetida**	**FIFO con zona repetida**	**LRU y FIFO con accesos uniformes**

10	32,2 %	30,5 %	3,4 %
25	69,2 %	59,5 %	8,6 %
50	83,3 %	75,0 %	17,0 %

6. **Limites conocidos:**
   - LRU solo gana cuando los accesos repiten zona. Con accesos uniformes empata con FIFO. Una cripta donde el jugador no vuelve atrás no se beneficiaría.
   - Insertar y eliminar en el vector es O(c). Con miles de fichas en memoria convendría un árbol balanceado.
   - La reconciliación recorre todas las fichas en uso en cada acción, aunque nada cambie. Con cientos de tipos distintos a la vez (12,5 % del límite con 400), habría que pasar a eventos precisos. En una partida normal son decenas.
   - Si todas las fichas están en uso, la caché supera su tope. Se eligió eso antes que perder una ficha que el juego sigue usando.
   - Se tomó como "enemigo vivo" a los activos y a los inactivos de la sala actual. Los de salas lejanas no cuentan y pueden salir de memoria.

4.5 Inventario

1. Problema

El jugador lleva objetos en el inventario, hasta **inventario_max**, en este caso se debe:

- Recorrer los objetos hacia adelante y hacia atrás desde donde está viendo.
- Soltar el objeto que está viendo, esté donde esté, con un costo que no dependa de su posición.
- Equipar un objeto lo pasa al frente del orden real.
- Ver los objetos ordenados por peso, valor o nombre sin cambiar el orden real.

Se midió qué operaciones se usan de verdad. En 4757 acciones al azar del jugador hubo 224 recogidas, 202 soltadas (veces que se sacó del inventario), 19 equipadas y unos 380 pasos de recorrido (siguiente y anterior). El inventario nunca tuvo más de 3 objetos a la vez. Es decir, las operaciones más comunes son recoger, soltar y recorrer, y equipar casi no pasa.

2. Alternativas

- **Arreglo:** es lo más simple, soltar o equipar en el medio obliga a mover todos los elementos que están después.
- **Lista enlazada simple:** soltar es rápido si ya tengo el nodo, pero no puedo ir hacia atrás sin recorrer desde el inicio, y el enunciado pide recorrer en las dos direcciones.
- **Lista doblemente enlazada con un cursor:** cada nodo conoce al anterior y al siguiente. El cursor marca el objeto que se está viendo.

3. Decisión

Se eligió la lista doblemente enlazada con cursor, ya que es la que cumple el enunciado sin trucos: avanzar, retroceder, soltar y equipar cuestan lo mismo sin importar dónde esté el objeto. Con un arreglo ese costo crece con el tamaño del inventario. La vista ordenada se hace sobre una copia de los objetos, así el orden real y el cursor no se tocan. Deshacer es lógico, al deshacer una suelta, el nodo se vuelve a poner junto a sus antiguos vecinos y así un cambio posterior no deja la lista dañada.

4. Análisis de complejidad

Operación	**Costo**
Avanzar o retroceder un objeto	O(1)
Recoger (agregar al final)	O(1)
Soltar el objeto actual	O(1)
Equipar (pasar al frente)	O(1)
Deshacer una de esas operaciones	O(1)
Vista ordenada (copiar y ordenar)	O(n)
Buscar un objeto o una llave	O(n)

5. Evidencia empírica

Se midió la alternativa descartada (el arreglo) contra la lista enlazada, con **python -m bench.bench_inventario**. Los tiempos son microsegundos por operación, con posiciones al azar:

```
Soltar el objeto que se ve y volver a recogerlo (microsegundos):
  objetos    lista enlazada     arreglo
       10              1.02        0.07
      100              1.03        0.10
     1000              1.07        0.32
    10000              1.27        1.66

Llevar al frente el objeto que se ve, como al equipar (microsegundos):
  objetos    lista enlazada     arreglo
       10              0.55        0.11
      100              0.73        0.13
     1000              0.58        0.89
    10000              0.73       12.18
```

La columna de la lista enlazada casi no cambia al crecer el inventario. La del arreglo sí crece. Para equipar, el cruce está entre 100 y 1000 objetos. Para soltar, entre 1000 y 10 000. Con pocos objetos, el arreglo es más rápido.

6. Límites conocidos

- Con los tamaños reales del juego, el arreglo habría sido igual o más rápido. Pero se eligió la lista enlazada porque el enunciado pide costo independiente de la posición, y porque así el tiempo no se descontrola si la cripta nueva tiene un **inventario_max** grande.
- Cada operación de la lista enlazada cuesta un poco más que la del arreglo, porque además prepara cómo deshacerse.
- Buscar un objeto o una llave recorre la lista completa (O(n)). Se usa poco y con inventarios chicos.
- Cada nodo ocupa más memoria que un elemento de un arreglo, por los dos enlaces.
- Los tiempos dependen de la computadora donde se midió. Lo que se mantiene es la forma: la lista enlazada se queda plana y el arreglo crece.

Botín de los enemigos: al morir, un enemigo deja en el suelo de su sala los objetos que indica su ficha (suelta). El modelo no conoce el catálogo, la fábrica le entrega a la partida una función que crea objetos nuevos a partir de un id y el modelo solo la llama. Así el modelo sigue sin depender de la API ni de los archivos. Lo que el enemigo suelta se registra en el historial, incluso un pergamino, de modo que al retroceder el botín desaparece y el enemigo revive; si no se registrara, retroceder dejaría objetos duplicados en el suelo.

4.6 Pergaminos de retroceso e historial reversible

1. Problema

El pergamino de retroceso tiene que devolver el juego al momento justo antes de la última acción del jugador. Eso incluye lo que pasó después: ataques de enemigos, efectos, el reloj, eventos de la agenda, rastro, vida, inventario, objetos del suelo, puertas y trampas. Solo se pueden deshacer las últimas 5 acciones. El enunciado prohíbe guardar copias completas del estado.

El pergamino no puede deshacerse a sí mismo. Si la cantidad de pergaminos se restaurara, una sola unidad serviría para retroceder para siempre.

2. Alternativas

- **Copiar el estado completo antes de cada acción:** es lo más simple de programar, pero el enunciado lo prohíbe. Igual se midió para ver cuánto costaría.
- **Una sola lista larga de cambios con marcas entre acciones:** funciona, pero para descartar la acción más antigua habría que sacar elementos del principio de la lista y mover todos los demás.
- **Un cuadro por acción, guardados en un buffer circular de 5:** cada cambio registra una función que lo invierte.

3. Decisión

Se seleccionó el cuadro por acción, cada vez que el jugador actúa se abre un cuadro y cada cambio que ocurre (hasta la siguiente decisión del jugador) guarda una función que lo deshace, por ejemplo: devolver la vida a 12 o reinsertar este objeto en el suelo. Los cuadros viven en un buffer circular de tamaño 5, cuando se llena, el cuadro más antiguo se reemplaza sin mover nada.

Para deshacer, se ejecutan las funciones del último cuadro al revés, de la última a la primera. También se guarda y se restaura el estado del generador de números al azar, para que lo que pase después del retroceso sea igual que antes.

Tres cosas quedan fuera a propósito: los pergaminos (el uso y la cantidad de pergaminos del jugador nunca se registran, en cambio, sí se registra que un enemigo suelte uno en el suelo), la bitácora de mensajes y el cursor del inventario, que no son estado del juego.

4. Complejidad

Operación	**Costo**
Abrir un cuadro o descartar el más antiguo	O(1)
Registrar un cambio	O(1)

Deshacer la última acción	O(k) con k=cambios de esa acción
Memoria	O(5*k) como mucho 5 cuadros
Copiar el estado completo (descartada)	O(tamaño del estado) en cada acción

5. Evidencia empírica

Se midió la alternativa descartada con **python -m bench.bench_historial**. Los tiempos son microsegundos por operación:

```
Historial de retroceso (microsegundos por operación)
escenario  salas  enemigos    acción  copiar todo  deshacer  registros
  pequeño     10        10      91.3        718.6      90.5     240 en 5
    larga    300        10     122.8      12375.1      92.5     240 en 5
   normal     20        30     336.2       1313.3     161.6     640 en 5
   grande    100       100    1237.0       4557.9     885.9    2040 en 5
  extremo    100       300    3848.4       9500.8    2839.7    6040 en 5
```

Comparando **pequeño** con **larga**: la cripta tiene 30 veces más salas y los mismos enemigos. Deshacer casi no cambia (90.5 y 92.5 microsegundos). Copiar todo se vuelve 17 veces más lento. En la cripta larga, copiar el estado tardaría unas 100 veces más que la propia acción, y habría que guardar 5 copias completas en memoria.

Además, la prueba aleatoria **tests/test_retroceso_aleatorio.py** compara una foto exacta del estado antes de una acción y después de retroceder, en muchas partidas al azar. Eso comprueba que el historial restaura bien lo que debe.

6. Límites conocidos

- Deshacer cuesta más cuando hay muchos eventos en una acción, en el escenario extremo (300 enemigos) llega a unos 2.8 ms. Sigue por debajo del límite de 20 ms por ciclo.
- Si algún cambio se hace sin registrarlo, el retroceso no lo deshace. La prueba aleatoria lo detecta solo para los datos que compara en su foto.
- La columna “Una acción” incluye el trabajo de los enemigos, no solo el costo del historial. No se midió una acción sin historial, así que no se puede dar el costo exacto de registrar.
- Cada cambio registrado ocupa memoria (una función por cambio), pero queda acotada por el límite de 5 cuadros.
- Los tiempos dependen de la computadora donde se midió. Lo que se mantiene es la forma: deshacer crece con los cambios, no con el tamaño de la cripta.

El pergamino no entra al historial: el uso del pergamino de retroceso no se guarda en el historial, así no se puede deshacer un retroceso ni encadenar usos infinitos.

4.7 El rastro del jugador

1. Problema

Cada sala recuerda el instante en que el jugador estuvo ahí por última vez. Un rastreador mira solo las salas vecinas conectadas por salidas abiertas y se mueve hacia la de rastro más fresco, siempre que tenga menos de 400 unidades de antigüedad. Si dos vecinas empatan, gana la de menor id.

2. Alternativas

- **Detectar la caducidad al consultar:** cada sala guarda un solo número (el instante) y el rastreador calcula reloj-rastro y descarta el que llegue a 400 o más.
- **Eliminar con un evento en la agenda:** cada vez que el jugador pasa por una sala se programa un evento llamado caduca el rastro a los 400 y se cancela el anterior de esa sala.
- **Barrer todas las salas:** en cada acción del jugador se recorren todas las salas y se borran los rastros viejos.

3. Decisión

Se seleccionó el detectar la caducidad al consultar, ya que era lo más barato, al jugador solo le cuesta anotar un número por movimiento, y al rastreador le cuesta mirar sus vecinas.

No mete eventos extra en la agenda, como con la segunda alternativa que habría un evento pendiente por cada sala que el jugador visitó y la agenda ya tiene mucho trabajo con los enemigos y los efectos.

Es sencillo de deshacer, porque el rastro es un solo valor por sala, así que el historial lo restaura con una asignación. Esta decisión es bastante funcional ya que, el rastreador solo recorre las salidas de su sala y no necesita conocer el mapa ni buscar rutas.

4. Complejidad

Operación	**Costo**
Anotar el rastro al moverse el jugador	O(1)
Decisión de un rastreador	O(k) con k=salidas de su sala (máximo 4)

Trabajo de varios rastreadores en un ciclo	O(k) para decidir, con k=rastreadores que actúan
Memoria del rastro	O(s), con s=número de salas, un número por sala

5. Evidencia empírica

Se midió con **python -m bench.bench_rastro**. Los tiempos son microsegundos por operación.

Decisión de un rastreador (el resto de la cripta es un recorrido largo con rastro):

```
 Rastro (microsegundos por operación)

Decisión de un rastreador (el resto de la cripta es un recorrido largo con rastro):
  salidas     10 salas    100 salas   1000 salas   5000 salas
        1         1.02         0.86         0.65         0.76
        2         1.46         0.60         0.57         0.55
        3         1.32         0.69         0.57         0.57
        4         0.79         1.24         1.28         1.27

Un ciclo con muchos rastreadores actuando (cripta de 100 salas):
 rastreadores       ciclo  por rastreador
           10       177.0           17.70
          100       757.9            7.58
         1000     13103.2           13.10

Costo de mantener el rastro en cada movimiento del jugador:
    salas  al consultar  con evento  con barrido
      100          0.20        5.37         1.52
     1000          0.23        7.04        22.25
     5000          0.23        6.09        93.39
```

Se muestran tres pruebas en microsegundos por operación. En la primera, la decisión de un rastreador tarda entre 0.55 y 1.5 microsegundos con 1 a 4 salidas y no sube cuando la cripta pasa de 100 a 5000 salas. Las diferencias pequeñas entre filas son ruido, y esto cumple lo que pide el enunciado: la decisión depende de las salidas de la sala, no del tamaño de la cripta.

En la segunda prueba, un ciclo con 10, 100 y 1000 rastreadores tarda 177, 758 y 13 103 microsegundos. El costo por rastreador no es constante, porque pasa de 17.7 a 7.58 microsegundos de 10 a 100 rastreadores y sube a 13.10 con 1000, porque cada acción pasa por la agenda de eventos y no por la decisión del rastro.

En la tercera prueba se midió el costo de mantener el rastro en cada movimiento del jugador. Anotar el instante al consultar cuesta unos 0.2 microsegundos sin importar el tamaño de la cripta. Programar un evento de caducidad por sala cuesta entre 5.4 y 7.0 microsegundos, y barrer todas las salas

llega a 93 microsegundos con 5000 salas. Por eso detectar la caducidad al consultar es la opción más barata y la única que no depende del tamaño de la cripta.

6. Límites conocidos

- El trabajo por ciclo crece un poco más que linealmente, ya que el costo por rastreador subió de 7.58 a 13.10 microsegundos al pasar de 100 a 1000 rastreadores. La decisión del rastreador sí es constante, pero cada acción pasa por la agenda, cuyo costo sube despacio con la cantidad de eventos. Es costo de la agenda, no del rastro.
- Las medidas de la tabla de decisión tienen ruido, sobre todo en la columna de 10 salas.
- La comparación con la segunda alternativa sólo mide programar y cancelar los eventos. No incluye ejecutarlos cuando llegan, ni deshacerlos con el historial. Con eso la segunda sería todavía más cara.
- Un rastro viejo sigue guardado en la sala hasta que el jugador vuelva a pasar, pero no se usa y ocupa un solo número por sala.
- El rastreador solo conoce a sus vecinas. Si ninguna tiene rastro fresco, espera, aunque el jugador esté cerca por otro camino. Es lo que define el enunciado.
- Los tiempos dependen de la computadora donde se midió.

4.8 Bitácora en pantalla

1. Problema

La pantalla muestra los últimos 20 mensajes del juego (ataques, movimientos, efectos, retrocesos). La operación más frecuente es agregar, porque casi cada cosa que pasa en el juego escribe un mensaje (en la simulación, una acción del jugador produce uno o más mensajes y cada acción de un enemigo también). Leer los 20 mensajes solo ocurre cuando se refresca la pantalla.

2. Alternativas

- **Guardar todos los mensajes y mostrar los últimos 20**: era lo más simple de realizar, pero el problema era que la memoria crece con la partida.
- **Lista de tamaño fijo que desplaza:** al llenarse, se saca el primero y los demás se corren una posición. La memoria es constante, pero desplaza físicamente los mensajes, que es justo lo que el enunciado prohíbe.
- **Buffer circular de 20 posiciones:** un arreglo fijo y un índice que dice dónde se escribe el siguiente. Al llenarse, el mensaje nuevo sustituye al más antiguo.

3. Decisión

Se escogió el buffer circular, tiene un arreglo de 20 posiciones y un número **siguiente** que indica dónde escribir. Para agregar, se escribe en esa posición y **siguiente** avanza uno, volviendo a 0 después de la 19. Nada se mueve de lugar, el mensaje más antiguo simplemente se sobrescribe. Para mostrar los mensajes, se leen empezando por **siguiente** que es el más antiguo y se da la vuelta al arreglo hasta llegar al más reciente.

La bitácora no entra al historial de retroceso, es un registro de lo que pasó (incluso el retroceso queda anotado), no estado del juego. Es una decisión que se tomó.

4. Complejidad

Operación	**Costo**
Agregar un mensaje	O(1)
Leer los 20 mensajes en orden	O(k) con k=20
Memoria	O(k) constante durante toda la partida

5. Evidencia empírica

Se midió con **python -m bench.bench_bitacora**. Además, **tests/test_bitacora.py** comprueba tres cosas: que sin llenarse devuelve todo en orden, que tras 53 mensajes conserva solo los últimos 20 en orden y que tras 1000 mensajes el arreglo sigue teniendo 20 posiciones.

```
 Bitácora

Memoria usada después de agregar N mensajes (KB):
  mensajes  buffer circular   guarda todo
      1000             2.6          98.7
    100000             2.6       10136.0
    500000             2.6       51704.7

Tiempo por mensaje agregado, con la bitácora llena (microsegundos):
 capacidad  buffer circular   desplazando
        20            0.328         0.196
       200            0.168         0.317
      2000            0.242         2.024
     20000            0.223        12.367
```

El buffer circular usa siempre unos 2.6 KB y guardar todo llega a unos 50 MB con 500 000 mensajes. Además, el buffer circular se queda en 0.2 a 0.3 microsegundos sin importar la capacidad. La lista que desplaza se vuelve cada vez más lenta: de 0.2 a 12.4 microsegundos al pasar de 20 a 20 000 posiciones.

6. Límites conocidos

- Con la capacidad real de 20, la lista que desplaza fue más rápida en esta medición, con tan pocos elementos desplazar no cuesta casi nada. De igual forma se eligió el buffer circular porque el enunciado prohíbe desplazar los mensajes y porque su costo no depende de la capacidad: la lista que desplaza ya cuesta 2 microsegundos con 2000 mensajes y 12 microsegundos con 20 000.
- Los mensajes viejos se pierden para siempre, porque se sobrescriben. Es lo que pide el enunciado, pero no sirve como registro completo de la partida.
- Leer los 20 mensajes arma una lista nueva cada vez, que cuesta O(20). Es poco, porque solo ocurre al refrescar la pantalla.
- Los tiempos dependen de la computadora donde se midió y de la ejecución: son diferencias de décimas de microsegundo, así que la comparación con 20 posiciones puede cambiar de una corrida a otra.

4.9 Ordenamiento

1. Problema

El programa tiene que ordenar listas en cuatro lugares: la tabla de puntajes, las vistas del inventario (por peso, valor o nombre), los índices del archivo de guardado y las listas auxiliares del catálogo. Los tamaños son muy diferentes según el contexto, el inventario tiene como máximo inventario_max objetos, mientras que la tabla de puntajes y las listas del catálogo pueden crecer. Además, ordenar es una vista que no cambia el orden real y no puede alterar el orden de los elementos iguales.

2. Alternativas

- **Solo insertion sort:** es simple y muy rápida con pocos elementos o con datos casi ordenados, pero con datos desordenados el tiempo crece con el cuadrado del tamaño.
- **Solo mergesort:** siempre trabaja en n log n, pero tiene más costo fijo con pocos elementos o con datos que ya vienen ordenados pierde.
- **Combinar las dos y escoger según los datos:** el insertion sort es rápido en datos casi ordenados y no usa memoria extra, y el mergesort tiene un tiempo garantizado pero usa memoria extra. Las dos son estables, o sea que los elementos con la misma clave conservan su orden original. Eso importa para las vistas del inventario, donde varios objetos pueden tener el mismo peso.

3. Decisión

Se decidió utilizar la combinación de ambas y la función **ordenar** escoge. Si la lista tiene 48 elementos o menos, usa insertion sort; si tiene más, hace una pasada contando los descensos, que son los pares seguidos fuera de orden. Si son

pocos se intenta la inserción con un límite de trabajo, básicamente si mueve más de 4*n elementos, se rinde. Y en cualquier otro caso (muchos descensos o la inserción se rindió) usa mergesort.

El límite existe por un caso trampa que se midió, dos mitades ordenadas, pero con la segunda mitad menor que la primera. Esa lista tiene un solo par fuera de orden, pero está muy desordenada, y el insertion sort tardaría mucho. Con el límite, el insertion sort se rinde pronto y el mergesort termina el trabajo.

4. Complejidad

Operación	**Costo**
Insertion sort	O(n²) en el peor caso; O(n + inversiones) en general, casi O(n) en datos casi ordenados
Mergesort	O(n log n) siempre, usa O(n) de memoria extra
Contar los descensos	O(n) una pasada
Insertion con límite	O(n) como mucho 4*n desplazamientos antes de rendirse
ordenar en el peor caso	O(n log n) contar+inserción limitada+mergesort

5. Evidencia empírica

Se midió con **python -m bench.bench_ordenamientos**.

```
  Ordenamientos (microsegundos por ordenamiento)

Datos al azar, por tamaño:
    n    insercion    mergesort  mejor
    4          0.9          2.7  insercion
    8          3.3          7.3  insercion
   16          7.1         16.7  insercion
   24         15.7         44.0  insercion
   32         51.6         75.3  insercion
   40         77.4         82.5  insercion
   48        119.3        113.5  mergesort
   64         86.3         93.7  insercion
   72        148.0        139.9  mergesort
   80        186.4        121.1  mergesort
   90        208.4        160.7  mergesort
  100        254.8        184.0  mergesort
  200       1092.3        364.3  mergesort
Mergesort empieza a ganar desde n = 48
```

Según cómo vienen los datos (ordenar = la selección automática del programa):

tipo de datos	n	insercion	mergesort	ordenar	mejor fijo
al azar	16	13.3	22.3	8.6	insercion
al azar	48	62.4	70.1	62.1	insercion
al azar	200	1134.2	339.5	419.0	mergesort
al azar	1000	33172.4	2039.7	2116.1	mergesort
casi ordenado	16	2.2	18.1	2.5	insercion
casi ordenado	48	6.6	54.8	6.1	insercion
casi ordenado	200	26.3	385.5	75.3	insercion
casi ordenado	1000	254.2	1961.9	204.2	insercion
ordenado	16	2.1	17.1	2.8	insercion
ordenado	48	7.3	88.3	7.7	insercion
ordenado	200	23.8	322.1	77.1	insercion
ordenado	1000	359.8	2173.3	253.9	insercion
inverso	16	14.8	27.2	16.8	insercion
inverso	48	102.7	56.0	123.6	mergesort
inverso	200	2004.0	442.0	361.8	mergesort
inverso	1000	61786.5	1928.3	1818.7	mergesort
dos mitades	16	9.8	17.0	11.7	insercion
dos mitades	48	63.6	61.7	91.7	mergesort
dos mitades	200	1423.0	433.7	526.0	mergesort
dos mitades	1000	28990.3	1704.0	2250.9	mergesort

Comparaciones costosas (datos al azar): veces que se pide la clave y tiempo con clave costosa:

n	llamadas ins	llamadas mer	ins (us)	mer (us)	mejor
8	31	30	12.9	21.6	insercion
16	88	96	33.1	52.8	insercion
32	314	244	121.1	167.7	insercion
48	626	424	243.6	195.0	mergesort
100	2570	1086	1127.7	470.7	mergesort
200	9471	2570	3887.1	1226.7	mergesort

Aquí se tienen tres pruebas en microsegundos por ordenamiento. En la primera, con datos al azar, la inserción gana con listas pequeñas y el mergesort gana desde unos 48 a 72. Por eso se fijó el límite en 48.

En la segunda prueba se midió cómo vienen los datos, con la columna **ordenar** que muestra lo que hace el programa. Con 1000 elementos casi ordenados, la inserción tarda unos 254 microsegundos y el mergesort unos 1962 microsegundos, ocho veces más, y **ordenar** queda en 204 microsegundos. Con datos al azar **ordenar** queda cerca del mergesort y con datos en orden inverso también. En el caso trampa de las dos mitades, **ordenar** tardó 2251 microsegundos, más que el mergesort, pero muy lejos de los 28 990 microsegundos que habría tardado la inserción sola.

En la tercera prueba se usó una clave costosa, la inserción pide la clave muchas más veces que el mergesort, con 200 elementos, 9471 llamadas contra 2570. Como consecuencia, con claves costosas el mergesort empieza a ganar antes

6. Límites conocidos

- El punto donde el mergesort empieza a ganar depende de la computadora y de la corrida: salió entre 48 y 72. Se dejó el 48 por ser el valor más seguro, pero habría que volver a medir en otra máquina.
- Con claves costosas el cruce baja a unos 32 a 48 elementos. Las claves que usa el programa son lecturas simples de un campo (peso, valor, nombre), así que no se ajustó. Si se ordenara por algo que cuesta calcular, habría que bajar el límite.
- Contar los descensos cuesta una pasada extra, con datos al azar de 200 elementos, **ordenar** tardó 419 contra 340 microsegundos del mergesort solo, un poco más. Con 1000 elementos la diferencia relativa fue menor, pero con datos casi ordenados de 200 elementos, **ordenar** tardó 75 contra 26 microsegundos de la inserción sola, pero lejos de los 386 microsegundos del mergesort.
- Hasta 48 elementos siempre se usa inserción. Con datos en orden inverso y 48 elementos, eso costó 124 contra 56 microsegundos del mergesort. Son pocos microsegundos, pero es el caso más desfavorable.

4.10 Guardado binario de la partida

1. **Problema**

Hay que guardar una partida en un archivo que detecte versiones incompatibles, permita recuperar una sala sin leer todo, y se pueda comparar contra JSON. El archivo guarda la semilla, las versiones de cripta y de catálogo, el contenido de las salas conocidas y la lista de comandos del jugador. Al cargar, el programa reconstruye la partida ejecutando esos comandos en silencio, apoyándose en el determinismo. Guardar ocurre cuando el jugador lo pide; cargar, cuando el jugador lo pide.

2. **Alternativas consideradas**
   - **JSON:** Un solo .json con todas las salas y comandos.
   - **Binario sin índice:** Utilizando registros seguidos. Para poder hallar una sala se recorren todos.
   - **Binario con índice al final:** Registros más una tabla de (id, posición, largo) ordenada por id.

3. **Decisión**

El archivo .sav es binario, de formato propio, con un índice al final que permite leer una sola sala sin recorrer el resto. Pesa la mitad que el mismo contenido en JSON, se guarda de 1,7 a 3,4 veces más rápido y una sala se lee de 2 a 5 veces más rápido; cargarlo todo da parejo con JSON.

El encabezado fijo ocupa 38 bytes: marca CRPT, versión del formato, semilla, cantidad de salas y de comandos, y las posiciones del índice y de los comandos.

Siguen tres cadenas con prefijo de largo (cripta, versión de cripta, versión de catálogo), luego un registro por sala, luego los comandos y al final el índice, con 16 bytes por sala. Los números se escriben con struct en little-endian. Para leer una sala se lee el encabezado, se carga el índice en un VectorBusqueda, se busca el id con búsqueda binaria y se hace seek al registro. Si la marca o la versión del formato no coinciden, GuardadoBinario lanza ErrorGuardado; además, validar_guardado rechaza cargar si la cripta, su versión o la del catálogo ya no son las mismas, y el juego sigue funcionando.

4. Analisis de complejidad

Con n salas guardadas y m comandos: guardar cuesta O(n + m) y escribe el índice al final, y después reescribe el encabezado con los valores reales. Leer una sala cuesta O(n) para cargar el índice (16 bytes por sala, no el archivo completo), O(log n) para buscar y una lectura del registro. Si el índice se conservara en memoria entre consultas, cada sala costaría O(log n).

5. Evidencia empírica

Salas	Tamaño bin / JSON	Guardar bin / JSON	Cargar todo bin / JSON	Una sala bin / JSON
50	10,2 / 17,7 KB	0,82 / 1,43 ms	0,35 / 0,28 ms	0,083 / 0,188 ms
500	56,8 / 115,0 KB	2,94 / 8,61 ms	3,15 / 2,08 ms	0,276 / 1,077 ms
5 000	540,7 / 1 112,6 KB	24,3 / 83,6 ms	30,7 / 29,5 ms	2,35 / 12,49 ms

```
Guardado binario vs JSON (misma información en ambos)
 salas | tamaño bin tamaño json | guardar bin guardar json | cargar todo bin cargar todo json | 1 sala bin 1 sala json
    50 |     10.2KB      17.7KB |      1.69ms       2.59ms |          0.61ms          0.50ms |    0.186ms     0.379ms
   500 |     56.8KB     115.0KB |      5.13ms      14.29ms |          4.84ms          3.74ms |    0.502ms     1.954ms
  5000 |    540.7KB    1112.6KB |     41.55ms     149.06ms |         51.21ms         52.25ms |    4.216ms    22.098ms
```

La ventaja del binario es el tamaño, guardar y, sobre todo, leer una sala, que en JSON obliga a interpretar todo el archivo. Cargar todo da parejo y con 50 y 500 salas gana JSON, porque json.load está escrito en C y nuestra lectura binaria está en Python.

6. Limites conocidos

- cargar hoy se apoya en la semilla y los comandos. La lectura de una sala por índice existe en el formato y está medida, pero el comando cargar no la necesita: reconstruye pidiendo o leyendo el contenido como en una partida normal.
- El índice se lee completo en cada consulta. Con decenas de miles de salas conviene conservarlo en memoria.
- Un cambio en el formato exige subir la versión; los archivos anteriores se rechazan, no se migran.
- Reconstruir por comandos restaura todo, incluidos los pergaminos de retroceso gastados antes de guardar. No guardamos una foto del estado del modelo.

4.11 Bitácora de partida y reproducción

1. **Problema**

Cada acción del jugador debe quedar registrada en un archivo de texto que solo crece, con un encabezado que permita saber contra qué datos se jugó. --replay partida.log debe reproducir la partida sin interacción, y si las versiones ya no coinciden debe decirlo en vez de reproducir contra datos distintos. La operación frecuente es anotar un comando, una vez por cada comando que el jugador escribe. Leer y reproducir ocurren una vez por ejecución de --replay.

2. **Alternativas consideradas**
   - **Foto del estado por acción:** Guarda una copia completa del estado tras cada comando
   - **Comandos sin vaciar el búfer:** Escribe sin forzar el disco
   - **Comandos con flush en cada linea:** Cada comando queda en disco al instante

3. **Decisión**

La bitácora guarda solo los comandos del jugador, unos 6 bytes cada uno, y la partida se reproduce volviendo a ejecutarlos con la misma semilla. Registrar cuesta lo mismo en cualquier momento de la partida; reproducir cuesta lo que costaron las acciones originales.

BitacoraPartida abre partida_<cripta>_<semilla>.log y escribe un encabezado de texto: la línea CRIPTA-LOG 1, el id de la cripta, la versión de la cripta, la versión del catálogo, la semilla y un separador ---. Después agrega un comando por línea y vacía el búfer en cada una. No se registran salir, guardar ni cargar. Para reproducir, LectorBitacora valida la primera línea y que el encabezado esté completo, y Aplicacion.reproducir compara las versiones del log con las del servicio. Si difieren, imprime que no puede garantizar una reproducción fiel y termina. Si coinciden, crea la partida con la semilla del log, la vista de reproducción

(que no pide nada) y ejecuta los comandos hasta que la partida termine. Todo el azar de Partida sale de random.Random(semilla), así que los mismos comandos dan la misma partida.

4. Complejidad algorítmica

Anotar un comando es O(1): una escritura y un flush. Leer el log es O(m) con m comandos. Reproducir cuesta m veces el costo de una acción del modelo, que en una cripta de 100 salas y 100 enemigos es de unos 0,6 ms.

5. Evidencia empírica

Comandos	Anotar con flush	Tamaño	Leer	Reejecutar
100	8,6 µs (incluye arranque)	0,6 KB	0,15 ms	69 ms
1 000	3,3 µs	5,7 KB	0,33 ms	668 ms
10 000	3,2 µs	56,7 KB	1,5 ms	6,0 s

Las mediciones varían dependiendo del equipo.

```
Anotar un comando (microsegundos) y tamaño del archivo:
 comandos  con flush  sin flush    tamaño  bytes/cmd
      100       49.4       3.96     0.6KB        6.6
     1000       27.9       2.61     5.7KB        5.9
    10000       17.8       1.36    56.7KB        5.8
```

La primera fila incluye el arranque (abrir el archivo); desde 1 000 comandos el costo se estabiliza en unos 3 µs. Sin flush serían unos 0,4 µs: la diferencia, 3 µs por comando, es el precio de no perder comandos si el programa se cae. Reejecutar cuesta unos 0,6 ms por comando, el costo de la acción del modelo.

6. Limites conocidos

- La reproducción exige que la versión de la cripta y la del catálogo coincidan exactamente con las del servicio. No se guardan copias de versiones anteriores, así que un log viejo no se puede reproducir.
- Reproducir es lineal en los comandos: una partida de 10 000 comandos tarda unos 6 s. No se puede saltar a un punto intermedio.

- Todo depende del determinismo. Cualquier azar o reloj real fuera de random.Random(semilla) rompe la reproducción fiel.
- Reproducir vuelve a pedir el contenido de las salas, porque el contenido no se guarda entre ejecuciones; las fichas del catálogo salen del disco.
- Un comando que el juego no reconoce también queda registrado; es inofensivo al reproducir.